Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Разработка программного обеспечения для визуализации кинематики шестереночных механизмов»

Студент ИУ7-52Б (Группа) _____ (Подпись, дата) М. А. Жаринов (И. О. Фамилия)

Руководитель курсовой работы _____ (Подпись, дата) Н. Н. Мартынюк (И. О. Фамилия)

2025 г.

РЕФЕРАТ

Расчётно-пояснительная записка 39 страниц, 9 рисунков, 4 таблицы, 10 источников.

КОМПЬЮТЕРНАЯ ГРАФИКА, КИНЕМАТИКА, ШЕСТЕРЕНОЧНЫЙ МЕХАНИЗМ, АЛГОРИТМ Z-БУФЕРА, ЗАКРАСКА ФОНГА, ПЛАНАРНЫЕ ТЕНИ, ГЕКСАГОНАЛЬНАЯ СЕТКА, C++, QT.

Объект исследования — методы визуализации трехмерных сцен и алгоритмы моделирования кинематических связей в механизмах.

Цель работы — разработка программного обеспечения для интерактивного конструирования и визуализации кинематики шестереночных механизмов в трехмерном пространстве.

В работе использованы методы аналитической геометрии и линейной алгебры для описания трехмерных объектов и их преобразований. Для визуализации применен метод программной растеризации с использованием алгоритма Z-буфера для удаления невидимых поверхностей. Расчет освещения выполнен с использованием модели Фонга. Для построения теней использован метод планарной проекции. Расчет кинематики базируется на представлении механизма в виде графа и алгоритме поиска в ширину (BFS).

Разработано программное обеспечение, позволяющее конструировать механизмы из зубчатых колес на гексагональной сетке, визуализировать их вращение с учетом физических связей и определять заклинивание. Реализовано корректное отображение освещения и теней.

Проведен анализ производительности системы. Установлено, что время отрисовки кадра линейно зависит от количества объектов в сцене. Программа обеспечивает интерактивную частоту кадров (около 18 FPS) при отрисовке 100 шестеренок.

СОДЕРЖАНИЕ

РЕФЕРАТ **4**

ВВЕДЕНИЕ **7**

1 Аналитический раздел **9**

1.1 Описание и формализация объектов сцены 9

1.2 Осевая система координат гексагональной сетки 9

1.3 Математическая модель кинематики механизмов 11

1.4 Математические основы построения трехмерного изображения 12

1.5 Анализ алгоритмов удаления невидимых поверхностей . . . 13

1.5.1 Алгоритм Робертса 13

1.5.2 Алгоритм художника 14

1.5.3 Алгоритм обратной трассировки лучей 14

1.5.4 Алгоритм, использующий Z-буфер 14

1.5.5 Сравнение алгоритмов удаления невидимых поверхностей 15

1.6 Анализ моделей освещения и закраски 15

1.6.1 Плоская закраска 15

1.6.2 Закраска Гуро 16

1.6.3 Закраска Фонга 16

1.6.4 Сравнение методов закраски 17

1.7 Анализ методов построения теней 17

1.7.1 Теневые карты 17

1.7.2 Теневые объемы 18

1.7.3 Планарная проекция 18

1.7.4 Сравнение методов построения теней 18

2 Конструкторская часть **20**

2.1 Требования к программному обеспечению 20

2.2 Разработка алгоритмов 21

2.2.1 Общая схема работы программы 21

2.2.2 Алгоритм растеризации треугольника 23

2.2.3 Алгоритм расчета кинематики 23

3 Технологическая часть **26**

3.1 Выбор средств реализации 26

3.2 Структура программного обеспечения 26

3.3 Реализация алгоритмов визуализации 29

3.4 Реализация алгоритма кинематики 30

3.5 Интерфейс пользователя 31

4 Исследовательская часть **33**

4.1 Цель исследования 33

4.2 Технические характеристики 33

4.3 Алгоритм проведения исследования 33

4.4 Результаты исследования 33

4.5 Анализ результатов 35

ЗАКЛЮЧЕНИЕ **36**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **38**

ПРИЛОЖЕНИЕ А **39**

ВВЕДЕНИЕ

Компьютерная графика играет ключевую роль в современном инженерном проектировании, образовании и моделировании физических процессов. Визуализация сложных механических систем, в том числе зубчатых передач, позволяет наглядно демонстрировать принципы их работы, выявлять ошибки проектирования на ранних этапах и облегчает понимание кинематических связей.

Статические чертежи и двухмерные схемы не всегда способны в полной мере передать динамику взаимодействия деталей, особенно когда речь идет о сложных цепочках зацепления. Построение интерактивных трехмерных моделей, позволяющих пользователю конструировать механизмы и наблюдать за их движением в реальном времени, является актуальной задачей.

Цель курсовой работы — разработка программного обеспечения для интерактивного конструирования и визуализации кинематики шестереночных механизмов в трехмерном пространстве.

Для достижения поставленной цели необходимо решить следующие **задачи**:

- выполнить анализ предметной области и существующих методов визуализации трехмерных сцен и моделирования кинематических связей;
- выбрать алгоритмы компьютерной графики для визуализации сцены: алгоритмы удаления невидимых поверхностей, методы закраски и построения теней;
- разработать выбранные алгоритмы визуализации;
- разработать алгоритм расчета кинематики зубчатых передач для корректного вычисления скоростей и направлений вращения всех элементов цепи;
- спроектировать структуру программного обеспечения и реализовать его;

— провести тестирование программы и оценить эффективность реализованных алгоритмов.

1 Аналитический раздел

В данном разделе проводится анализ предметной области, формализуются объекты сцены и рассматриваются математическая модель расчета кинематики шестереночных механизмов. Также производится обзор и сравнительный анализ алгоритмов компьютерной графики для выбора наиболее подходящих алгоритмов в рамках поставленной задачи.

1.1 Описание и формализация объектов сцены

Сцена представляет собой трехмерное пространство, содержащее следующие типы объектов.

1) Шестерни (зубчатые колеса). Являются основными динамическими объектами. Модель состоит из цилиндрического основания и зубьев, расположенных по окружности. Важными параметрами для построения являются:

   — R — радиус делительной окружности;

   — N — количество зубьев (пропорционально радиусу для обеспечения зацепления);

   — h — толщина шестерни;

   — ϕ — текущий угол поворота вокруг собственной оси.

2) Основание (пол) — плоскость, на которую проецируются тени объектов. На основании нанесена гексагональная сетка для удобства позиционирования.

3) Источник света — бесконечно удаленный объект, задается углом падения и направлением (азимутом) света.

1.2 Осевая система координат гексагональной сетки

Для позиционирования объектов используется гексагональная сетка. В отличие от прямоугольной сетки, где каждый узел имеет 4 соседа, в

гексагональной сетке каждый узел имеет 6 равноудаленных соседей. Эта сетка лучше подходит для размещения шестеренок, так как в данной сетке отсутствуют диагонали. В отличие от прямоугольной сетки, где оси ортогональны, в гексагональной структуре естественным образом выделяются три оси симметрии. Для математического описания такой структуры наиболее удобна кубическая система координат (x, y, z), подчиняющаяся условию $x + y + z = 0$. Однако, поскольку это условие делает одну из координат избыточной, для хранения данных и оптимизации вычислений применяется осевая система координат [1].

В осевой системе положение любой ячейки (шестерни) задается парой целочисленных координат (q, r):

— q — координата, соответствующая столбцу (направлена под углом 30° к горизонтали);

— r — координата, соответствующая ряду (направлена вертикально или под углом, в зависимости от ориентации).

Третья (кубическая) координата s при необходимости может быть вычислена неявно по формуле (1):

$$s = -q - r. \tag{1}$$

Данная система координат позволяет выполнять операции смещения и поиска соседей с помощью простого сложения векторов, избегая сложных проверок четности строк, свойственных системам со смещением.

Для визуализации сцены необходимо преобразовать дискретные координаты сетки (q, r) в непрерывные мировые координаты (x_{world}, z_{world}). В данной работе используется ориентация шестиугольников вершиной вверх. Преобразование осуществляется по формулам (2).

$$\begin{cases} x_{world} = S \cdot \sqrt{3} \cdot \left(q + \frac{r}{2}\right), \\ z_{world} = S \cdot \frac{3}{2} \cdot r, \end{cases} \tag{2}$$

где S — размер шестиугольника (радиус описанной окружности, расстояние от центра до вершины).

Обратное преобразование (нахождение ячейки (q, r) по клику мыши в точку (x, z)) выполняется путем инверсии матрицы преобразования (2) и последующего округления до ближайшего узла сетки по формуле (3).

$$\begin{cases} q_{float} = \frac{\frac{\sqrt{3}}{3} \cdot x_{world} - \frac{1}{3} \cdot z_{world}}{S}, \\ r_{float} = \frac{\frac{2}{3} \cdot z_{world}}{S}. \end{cases} \tag{3}$$

После получения вещественных координат (q_{float}, r_{float}) применяется алгоритм округления кубических координат для устранения погрешностей дискретизации и выбора корректной ячейки.

1.3 Математическая модель кинематики механизмов

Система сцепленных шестеренок может быть представлена в виде неориентированного графа $G = (V, E)$, где V — множество шестеренок (вершин), а E — множество пар сцепленных шестеренок (ребер).

В системе присутствует один источник вращения (ведущая шестерня) с заданной угловой скоростью ω_{driver}. Задача кинематического расчета сводится к определению угловых скоростей ω_i для всех остальных шестеренок.

Условие отсутствия проскальзывания в точке контакта двух шестеренок i и j требует равенства линейных скоростей их зубьев v_i и v_j:

$$v_i = -v_j. \tag{4}$$

Знак минус обусловлен тем, что в паре внешнего зацепления шестерни вращаются в противоположных направлениях.

Связь линейной скорости v и угловой скорости ω выражается формулой $v = \omega \cdot R$. Следовательно, передаточная функция для двух сцепленных шестеренок с угловыми скоростями ω_i и ω_j и радиусами R_i и R_j описывается уравнением (5) [2].

$$\omega_j = -\omega_i \cdot \frac{R_i}{R_j}. \tag{5}$$

Для распространения вращения по всей системе используется алгоритм обхода графа в ширину (BFS). Алгоритм также должен выполнять проверку на заклинивание. Заклинивание происходит, если в графе су-

ществуют циклы, и при обходе обнаруживается, что для одной и той же шестерни приходят противоречивые требования к скорости или направлению вращения от разных соседей. Если для вершины k уже вычислена скорость ω_k, а новый путь от соседа m требует скорости ω'_k, и $|\omega_k - \omega'_k| > \varepsilon$, то система считается заклинившей.

Также для визуальной корректности необходимо учитывать фазовое смещение θ, чтобы зубья одной шестерни попадали во впадины другой. Угол поворота ведомой шестерни радиусом R_{next} и с количеством зубьев N_{next} рассчитывается по формуле (6).

$$\theta_{next} = \alpha \cdot (1 + \frac{R_{curr}}{R_{next}}) - \theta_{curr} \cdot \frac{R_{curr}}{R_{next}} + \pi + \frac{\pi}{N_{next}}, \tag{6}$$

где

— α — угол вектора, соединяющего центры шестеренок;

— θ_{curr} — фазовое смещение текущей шестерни;

— θ_{next} — фазовое смещение ведомой шестерни;

— R_{curr} — радиус текущей шестерни.

1.4 Математические основы построения трехмерного изображения

Для визуализации трехмерной сцены на двумерном экране используется графический конвейер, включающий последовательность преобразований систем координат [3].

Любая точка P_{local} из локального пространства модели преобразуется в экранные координаты P_{screen} с помощью умножения на матрицы по формуле (7).

$$P_{screen} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot P_{local}, \tag{7}$$

где

1) Модельная матрица M_{model} отвечает за положение, поворот и масштаб объекта в мировом пространстве. Для вращающейся шестерни эта матрица пересчитывается каждый кадр в зависимости от угла ϕ.

2) Видовая матрица M_{view} трансформирует мировые координаты в систему координат камеры. Она определяется позицией наблюдателя, точкой, на которую он смотрит, и вектором «вверх».

3) Проекционная матрица M_{proj} преобразует усеченную пирамиду видимости в куб нормализованных координат устройства. Для реалистичного отображения используется перспективная проекция, при которой удаленные объекты кажутся меньше. Элементы матрицы зависят от угла обзора и соотношения сторон экрана.

После матричных преобразований производится перспективное деление (деление на координату w) и переход к экранным координатам.

1.5 Анализ алгоритмов удаления невидимых поверхностей

При отображении трехмерных объектов на двумерном экране необходимо определить, какие части объектов видны наблюдателю, а какие скрыты другими объектами или собственными гранями. Рассмотрим основные методы решения этой задачи.

1.5.1 Алгоритм Робертса

Данный алгоритм работает в пространстве объекта и предназначен преимущественно для выпуклых многогранников. Алгоритм состоит из двух этапов.

1) Удаление нелицевых граней. Для каждой грани вычисляется скалярное произведение вектора взгляда и внешней нормали к грани. Если результат положительный, грань «отвернута» от наблюдателя и отбрасывается.

2) Сравнение каждого ребра каждого тела с остальными телами. Проверяется, экранируется ли ребро (или его часть) каким-либо другим объектом сцены.

Алгоритм математически точен, но имеет сложность $O(N^2)$, где N — количество объектов, и плохо масштабируется для сложных сцен с невыпуклыми

объектами (требует разбиения на выпуклые части) [4].

1.5.2 Алгоритм художника

Алгоритм работает в пространстве объекта и изображения. Суть заключается в сортировке всех полигонов сцены по их глубине (расстоянию от наблюдателя). Отрисовка происходит в порядке от самых дальних полигонов к самым ближним. Ближние объекты перекрывают дальние в буфере кадра. Главный недостаток алгоритма — невозможность корректной обработки случаев циклического перекрытия (полигон A перекрывает B, B перекрывает C, C перекрывает A) и случаев взаимопересечения полигонов без их предварительного геометрического разбиения [4].

1.5.3 Алгоритм обратной трассировки лучей

Метод работает в пространстве изображения. Для каждого пикселя экрана из центра проекции (глаза наблюдателя) выпускается луч. Находится ближайшая точка пересечения луча с объектами сцены. Цвет пикселя определяется свойствами объекта в точке пересечения. Алгоритм автоматически решает проблему видимости, теней и отражений. Однако он требует больших вычислительных затрат, так как поиск пересечений должен выполняться для каждого пикселя с каждым полигоном [5].

1.5.4 Алгоритм, использующий Z-буфер

Работает в пространстве изображения. Помимо буфера цвета, используется буфер глубины (Z-Buffer) той же размерности. В ячейках Z-буфера хранятся значения глубины (координаты z или $1/z$) ближайшего к наблюдателю объекта, спроецированного в данный пиксель [6]. Алгоритм состоит из следующих этапов.

1) Изначально Z-буфер заполняется значением бесконечности (максимальной глубины).

2) При растеризации каждого полигона для каждого пикселя вычисляется его глубина $z_{current}$.

3) Сравнивается $z_{current}$ со значением z_{buffer}, уже хранящимся в буфере.

4) Если $z_{current} < z_{buffer}$, то пиксель ближе к наблюдателю: цвет записывается в буфер кадра, а z_{buffer} обновляется значением $z_{current}$.

1.5.5 Сравнение алгоритмов удаления невидимых поверхностей

Сравнение алгоритмов удаления невидимых поверхностей представлено в таблице 1, где N — количество объектов, n — количество пикселей.

Таблица 1 – Сравнение алгоритмов удаления невидимых поверхностей

Алгоритм	Сложность	Использование памяти	Ограничения
Робертса	$O(N^2)$	$O(N)$	Только выпуклые тела, сложность реализации
Художника	$O(N \log N)$	$O(N)$	Циклические перекрытия, пересечения
Трассировка лучей	$O(n \cdot N)$	$O(N)$	Очень низкая скорость без оптимизации
Z-буфер	$O(n \cdot N)$	$O(N + n)$	Требует доп. памяти, возможен Z-fighting

Для выполнения работы выбран алгоритм, использующий Z-буфер. Он позволяет корректно отображать пересекающиеся объекты (ось, проходящая сквозь шестерню), прост в реализации при программной растеризации и имеет линейную зависимость времени работы от количества обрабатываемых фрагментов.

1.6 Анализ моделей освещения и закраски

Для придания трехмерным объектам объема и реалистичности необходимо рассчитывать взаимодействие света с поверхностью.

1.6.1 Плоская закраска

Вектор нормали вычисляется один раз для всего полигона (как векторное произведение двух его сторон). Освещенность рассчитывается для этого вектора, и весь полигон закрашивается одним цветом. Метод очень быстр,

но дает изображение с четко видимыми гранями («фасеточная» структура). Это подходит для стилизованной графики, но плохо передает кривизну поверхностей [5].

1.6.2 Закраска Гуро

Нормали задаются в вершинах полигона. Освещенность рассчитывается для каждой из трех вершин. Затем полученные цвета линейно интерполируются по поверхности треугольника в процессе растеризации. Метод позволяет сгладить грани, но имеет существенный недостаток: зеркальные блики рассчитываются некорректно. Если блик попадает в центр полигона, а вершины освещены слабо, блик просто исчезнет [7].

1.6.3 Закраска Фонга

Наиболее качественный метод локального освещения. В вершинах задаются нормали. В процессе растеризации между вершинами интерполируются векторы нормалей. Освещение рассчитывается для каждого пикселя на основе интерполированной нормали. Модель освещения Фонга включает три компоненты и считается по формуле (8) [4].

$$I = I_{ambient} + I_{diffuse} \cdot (\vec{L} \cdot \vec{N}) + I_{specular} \cdot (\vec{R} \cdot \vec{V})^n, \tag{8}$$

где

— $I_{ambient}$ — фоновое освещение;

— $I_{diffuse}$ — рассеянное освещение;

— $I_{specular}$ — зеркальное освещение;

— n — показатель гладкости материала;

— $\vec{N}$ — нормаль;

— $\vec{L}$ — вектор на свет;

— $\vec{R}$ — вектор отражения;

— $\vec{V}$ — вектор на наблюдателя.

Метод дает качественные блики на криволинейных поверхностях.

1.6.4 Сравнение методов закраски

Пример работы методов закраски представлен на рисунке 1.

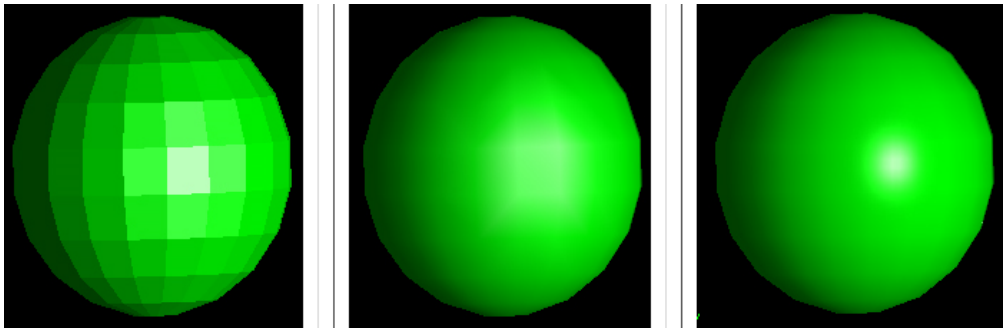

Рисунок 1 – Пример работы методов закраски: слева — плоская, в центре — Гуро, справа — Фонг

Сравнение методов закраски представлено в таблице 2.

Таблица 2 – Сравнение методов закраски

Метод	Качество бликов	Вычислительные затраты
Плоская	Отсутствуют	Минимальные
Гуро	Низкое	Средние
Фонга	Высокое	Высокие

Для выполнения работы выбрана закраска Фонга, так как она позволяет максимально реалистично отображать блики.

1.7 Анализ методов построения теней

Тени являются важнейшим пространственным ориентиром, позволяющим оценить положение объектов.

1.7.1 Теневые карты

Алгоритм требует двух проходов рендеринга. Сначала сцена рендерится с точки зрения источника света, сохраняя только глубину. Затем, при основном рендеринге, координаты каждого пикселя преобразуются в пространство света и сравниваются с картой глубины. Метод универсален (тени падают на любые объекты), но подвержен артефактам дискретизации («лесенки» на краях теней) и сложен в реализации [7].

1.7.2 Теневые объемы

Геометрический метод, основанный на построении усеченных пирамид, образованных силуэтом объекта и источником света. Использует буфер трафарета для подсчета вхождений пикселя в объем тени. Дает идеально четкие края, но требует больших затрат на геометрические вычисления и сложен для моделей с большим количеством полигонов [6].

1.7.3 Планарная проекция

Метод применим, когда тени нужно отбрасывать только на плоскую поверхность. Геометрия объекта «сплющивается» на плоскость пола с помощью матрицы проекции M_{shadow}, которая зависит от положения источника света и уравнения плоскости. Сплющенная геометрия отрисовывается темным цветом без записи в Z-буфер. Матрица проекции на плоскость $y = 0$ для направленного света $\vec{L}$ вычисляется по формуле (9).

$$M_{shadow} = \begin{pmatrix} 1 & -L_x/L_y & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -L_z/L_y & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{9}$$

1.7.4 Сравнение методов построения теней

Сравнение методов построения теней представлено в таблице 3

Таблица 3 – Сравнение методов построения теней

Метод	Область применения	Качество краев	Производительность
Теневые карты	Универсальный	Низкое	Зависит от разрешения карты
Теневые объемы	Универсальный	Высокое	Низкая
Планарная проекция	Только на плоскость	Высокое	Высокая

Для выполнения работы выбран метод планарной проекции. В задаче визуализации настольного механизма тени падают исключительно на

плоскость основания (стол). Этот метод обеспечивает высокую производительность и отличное качество краев тени.

Вывод

На основе проведенного анализа для реализации выбраны следующие методы:

— Удаление невидимых поверхностей: Алгоритм Z-буфера, обеспечивающий корректную обработку пересекающейся геометрии.

— Освещение: Модель Фонга с попиксельным расчетом для качественной передачи металлических бликов.

— Тени: Планарная проекция на плоскость основания, как наиболее эффективный метод для данной постановки задачи.

— Кинематика: Графовый подход с поиском в ширину для распространения вращения и детектирования коллизий.

2 Конструкторская часть

В данном разделе формулируются требования к программному обеспечению, описывается общая структура программы и детально рассматриваются алгоритмы, лежащие в основе визуализации и моделирования кинематики.

2.1 Требования к программному обеспечению

Разрабатываемое программное обеспечение предназначено для моделирования и визуализации работы шестереночных механизмов. К программе предъявляются следующие функциональные требования:

1) программа должна предоставлять графический интерфейс пользователя для взаимодействия со сценой;

2) должна быть реализована возможность добавления шестеренок различных размеров на гексагональную сетку;

3) должна быть предусмотрена возможность удаления установленных шестеренок;

4) программа должна обеспечивать визуализацию трехмерной сцены, включающей пол, сетку, оси и вращающиеся шестерни;

5) должна быть реализована возможность управления камерой (вращение вокруг центра сцены, масштабирование);

6) программа должна моделировать кинематику механизма: передачу вращения от ведущей шестерни к ведомым с учетом передаточных чисел;

7) должен быть реализован контроль коллизий: запрет установки шестеренок в пересекающиеся позиции;

8) должна быть предусмотрена индикация заклинивания механизма при создании противоречивых кинематических связей;

9) программа должна позволять управлять скоростью и направлением вращения ведущей шестерни.

2.2 Разработка алгоритмов

Для реализации поставленных задач были разработаны алгоритмы визуализации и расчета физики сцены.

2.2.1 Общая схема работы программы

Работа программы строится на основе главного цикла обработки событий и таймера обновления кадров. Схема алгоритма работы программы представлена на рисунке 2.

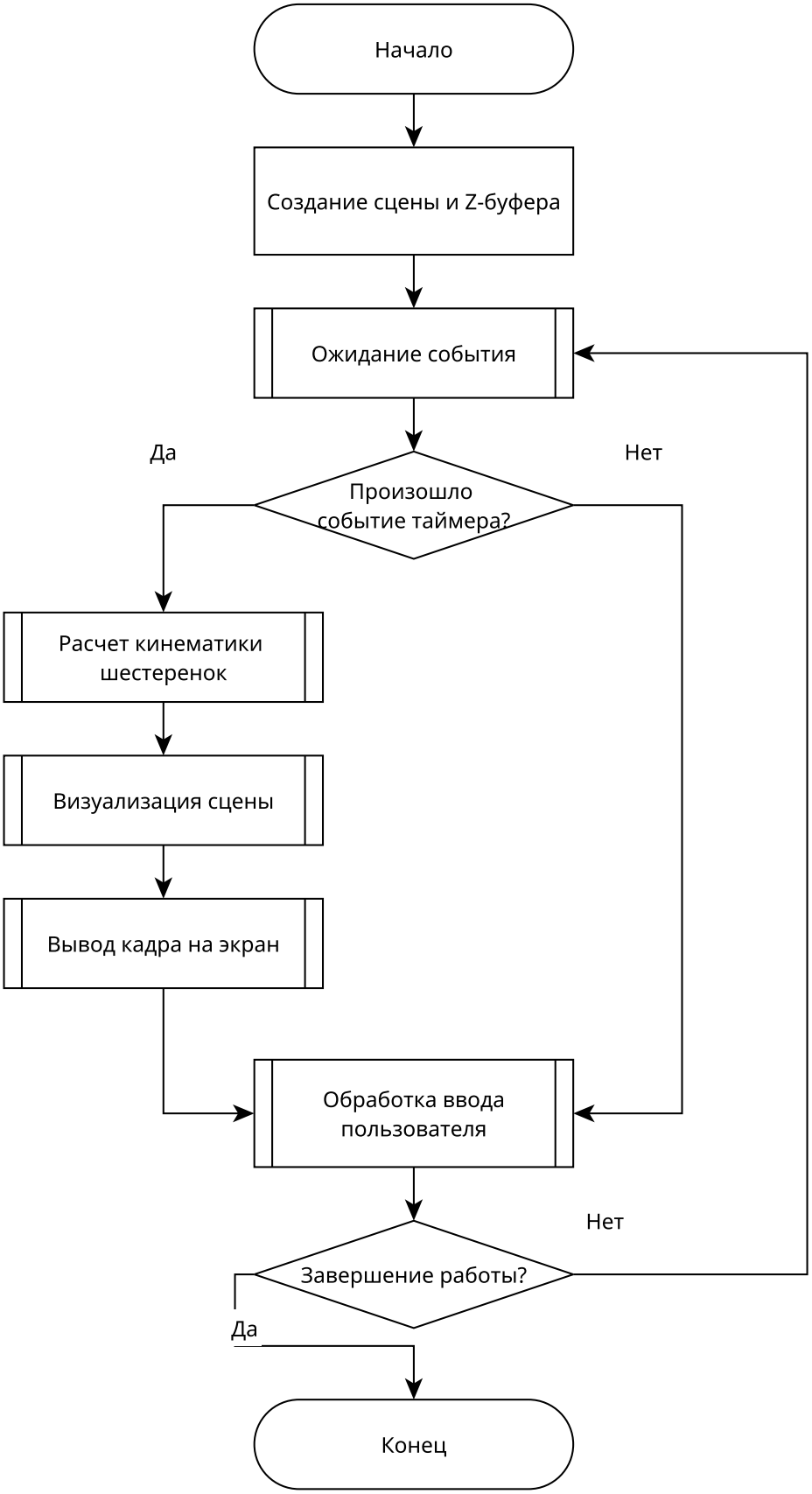

Рисунок 2 – Схема алгоритма работы программы

2.2.2 Алгоритм растеризации треугольника

Центральным элементом графического конвейера является алгоритм растеризации треугольника с использованием Z-буфера и интерполяцией атрибутов для закраски Фонга. Схема алгоритма представлена на рисунке 3.

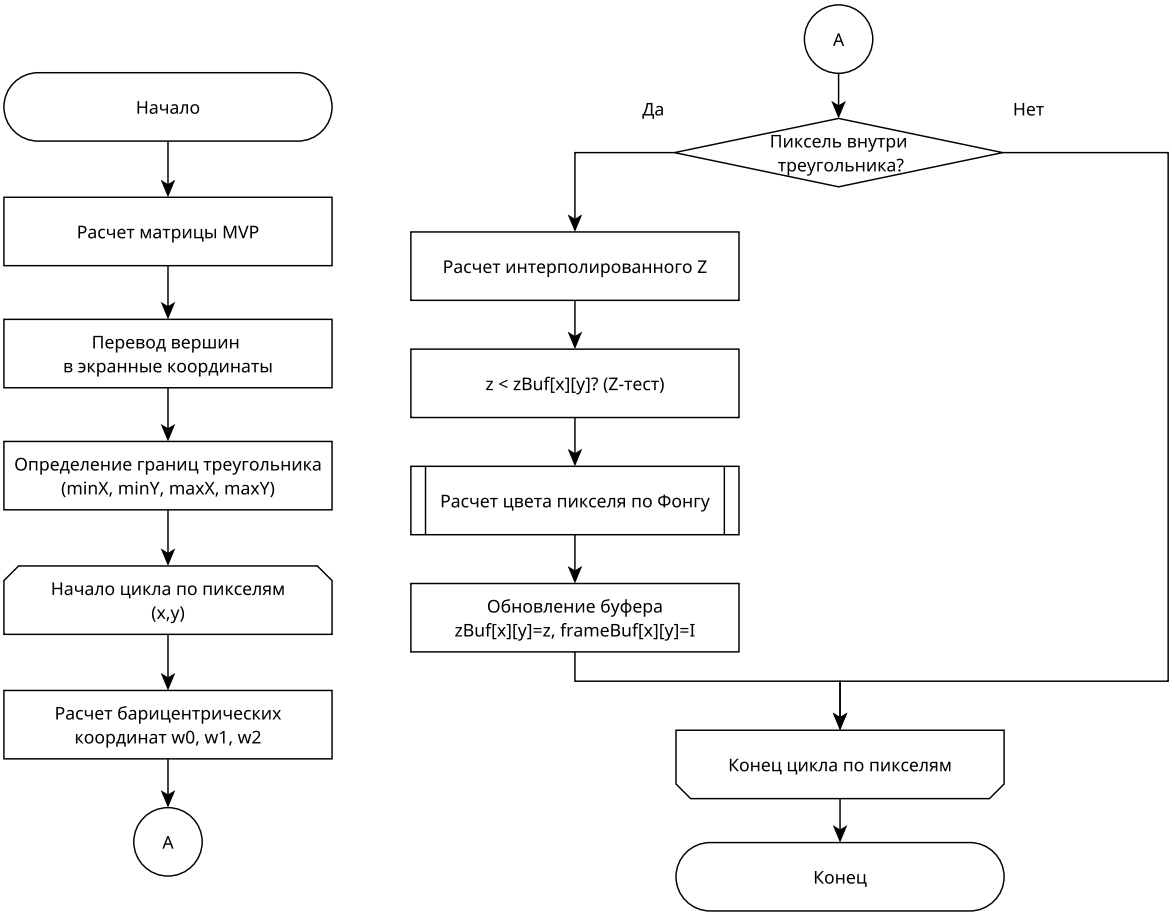

Рисунок 3 – Схема алгоритма растеризации треугольника

2.2.3 Алгоритм расчета кинематики

Для определения скоростей вращения всех шестеренок в механизме используется алгоритм обхода графа в ширину (BFS). Схема алгоритма представлена на рисунках 4 и 5.

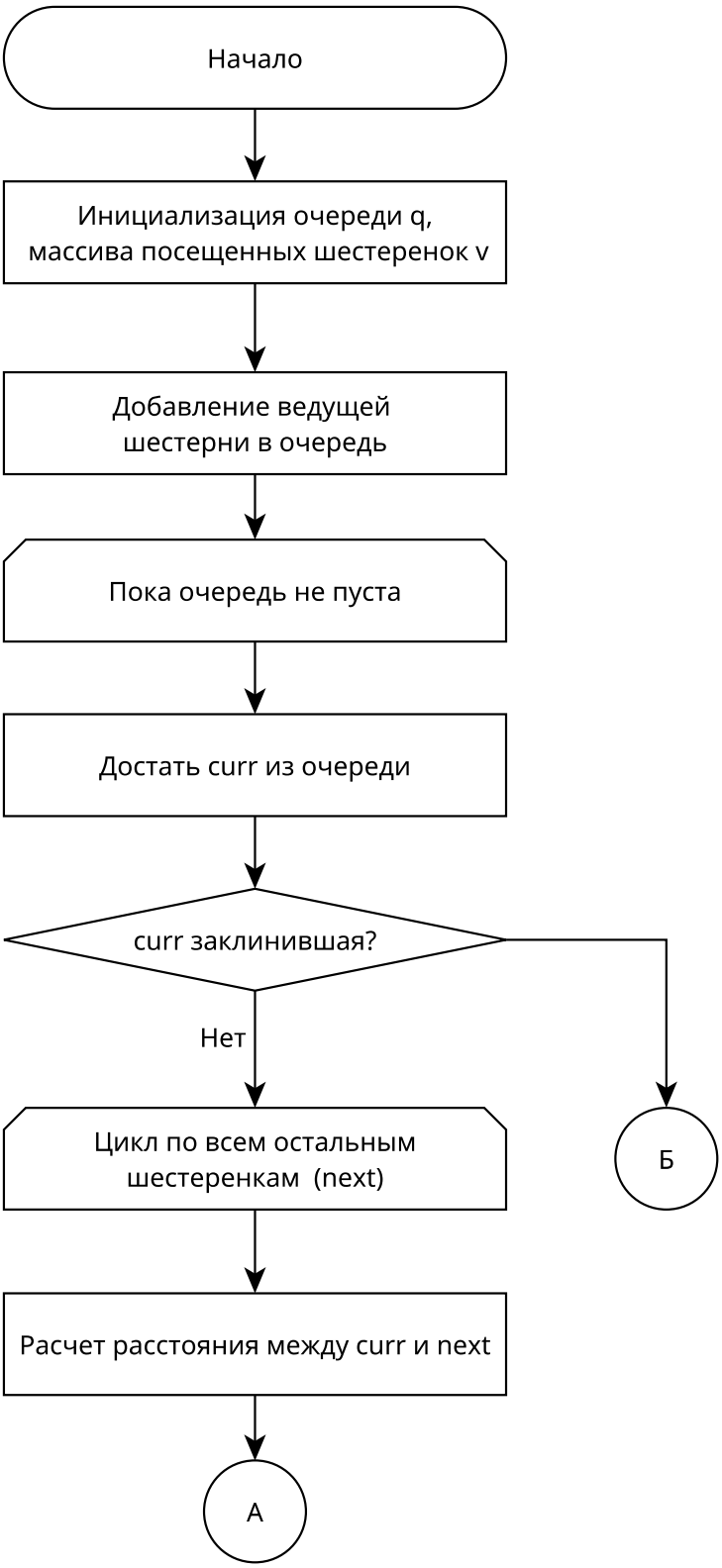

Рисунок 4 – Схема алгоритма расчета кинематики 1

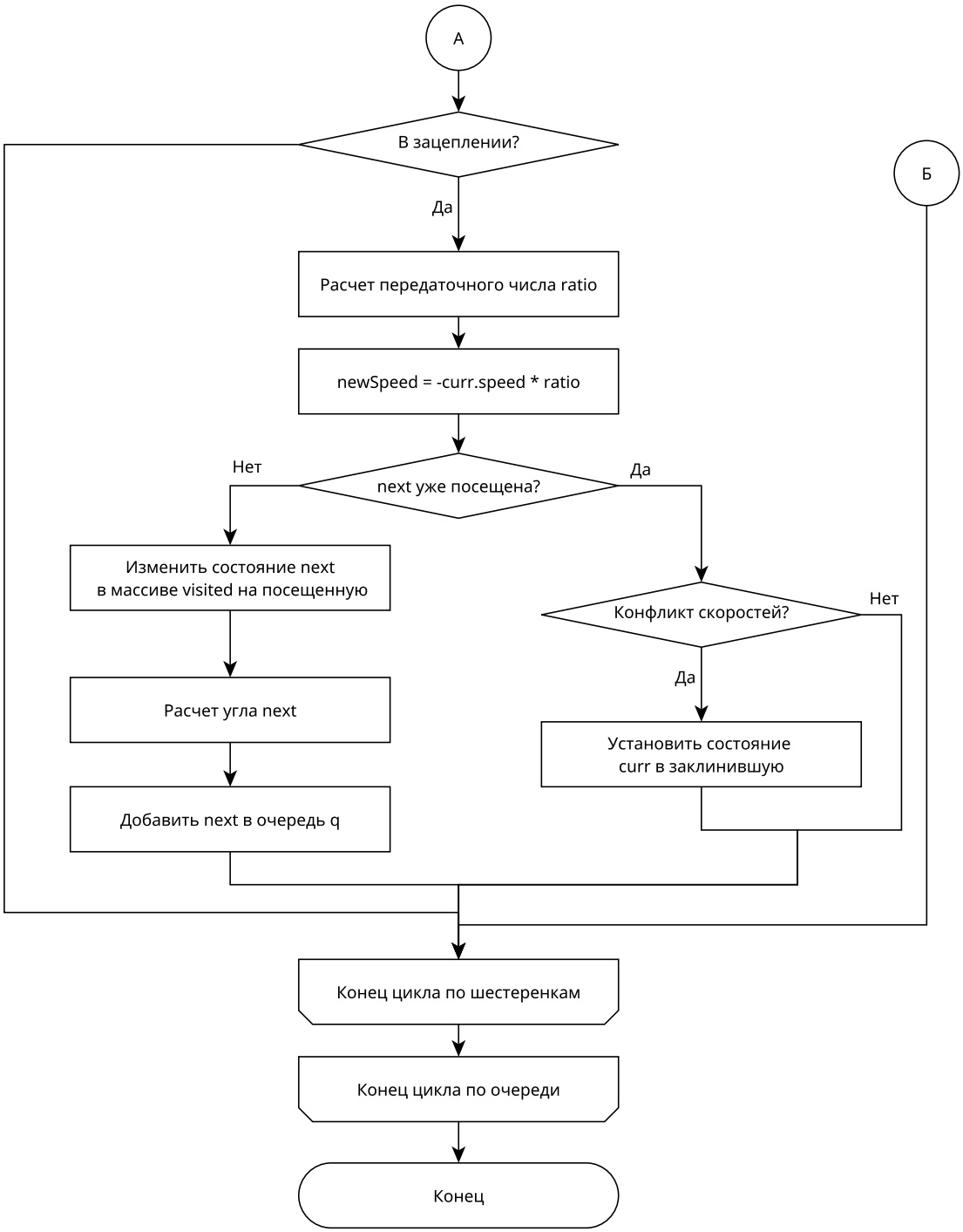

Рисунок 5 – Схема алгоритма расчета кинематики 2

Вывод

В данном разделе были сформулированы требования к программному обеспечению, включающие функциональные возможности визуализации и управления сценой. Описаны основные алгоритмы визуализации и расчета кинематики.

3 Технологическая часть

В данном разделе обосновывается выбор средств реализации, приводится описание структуры разработанного программного обеспечения, а также представлены листинги реализации ключевых алгоритмов визуализации и расчета кинематики.

3.1 Выбор средств реализации

Для разработки программы был выбран язык программирования C++ [8] (стандарт C++20). В качестве графической библиотеки и фреймворка для создания пользовательского интерфейса использована библиотека Qt [9] (версия 6). Она предоставляет:

— кроссплатформенные средства создания оконного интерфейса (модуль `QtWidgets`);

— оптимизированные математические классы (`QVector3D`, `QMatrix4x4`);

— класс `QImage` для удобного вывода сформированного буфера кадра на экран.

Среда разработки — Zed editor, система сборки — qmake.

3.2 Структура программного обеспечения

Программа реализована с использованием принципов объектно-ориентированного программирования. Архитектура приложения включает следующие основные классы и структуры:

— `MainWindow` — класс главного окна приложения. Отвечает за инициализацию графического интерфейса пользователя и обработку сигналов от элементов управления (кнопок, слайдеров, чекбоксов).

— `RenderWidget` — класс виджета для вывода трехмерной графики. Реализует цикл с использованием таймера, обрабатывает события ввода (нажатия клавиш мыши, перемещение курсора, прокрутку колеса) и управляет взаимодействием между пользователем и сценой.

— `Renderer` — класс, инкапсулирующий логику программного рендеринга. Содержит методы для:

  – трансформации вершин из локального пространства в экранное;
  – растеризации треугольников с использованием барицентрических координат;
  – управления Z-буфером для удаления невидимых поверхностей;
  – расчета освещения по модели Фонга для каждого пикселя;
  – отрисовки теней методом планарной проекции.

— `Scene` — класс, описывающий состояние виртуального мира. Хранит контейнер объектов (шестеренок), параметры гексагональной сетки и геометрию статического окружения (пола). Реализует алгоритмы кинематики: поиск ведущей шестерни, распространение вращения по графу сцеплений и детектирование заклинивания механизма.

— `Camera` — класс виртуальной камеры. Отвечает за формирование матриц вида и проекции. Реализует управление положением наблюдателя в сферических координатах (азимут, угол места, радиус) и предоставляет функционал для генерации лучей из экранных координат в мировое пространство.

— `Gear` — класс, описывающий отдельную шестеренку. Содержит параметры объекта (радиус, количество зубьев, текущий угол поворота) и методы для процедурной генерации полигональной сетки (вершин и нормалей) тела шестерни и её оси вращения.

— `Geom.h` — заголовочный файл, содержащий вспомогательные структуры данных: `Vertex` (вершина с координатами и нормалью) и `Triangle` (набор из трех вершин и цвета).

Диаграмма классов представлена на рис. 6.

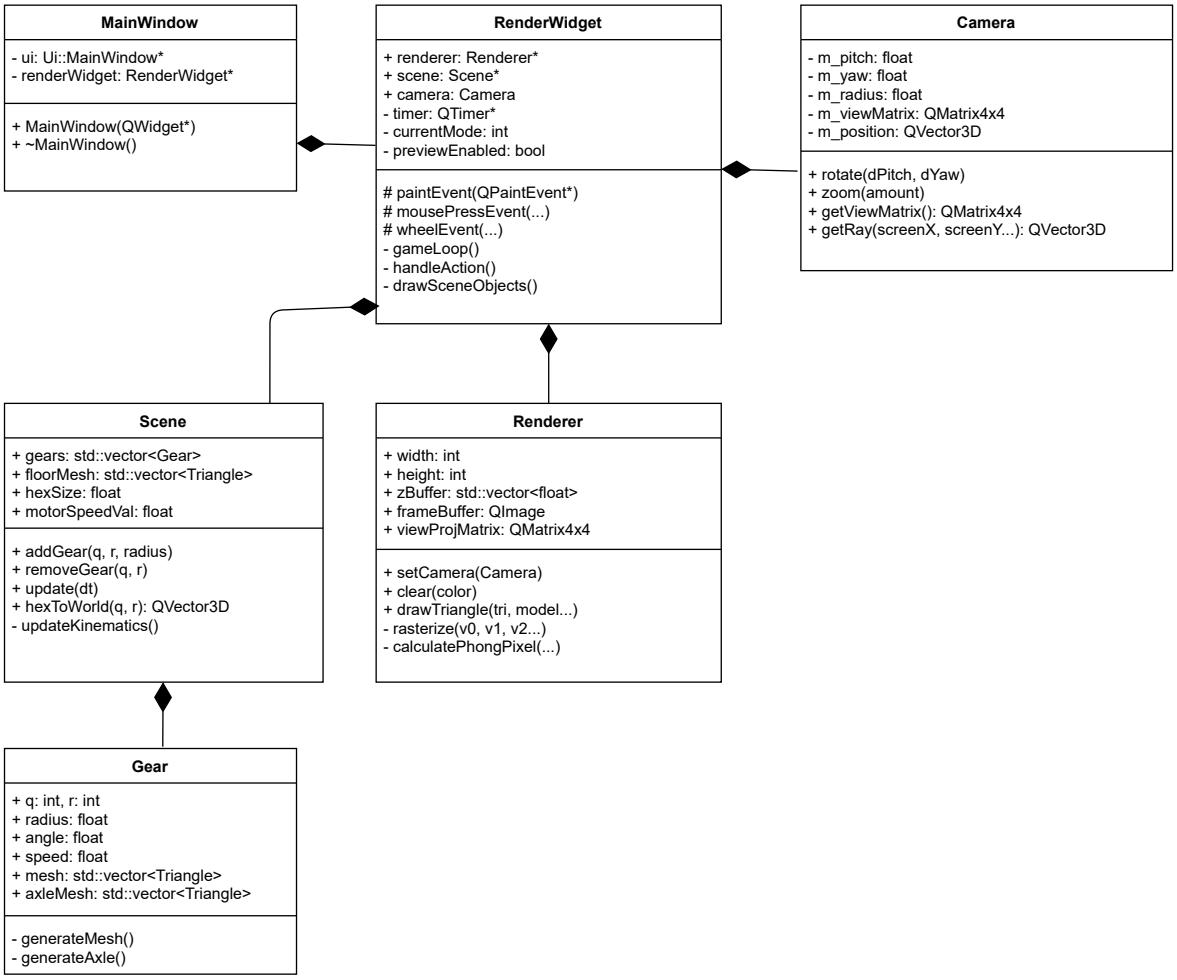

Рисунок 6 – Диаграмма классов

Диаграмма компонентов представлена на рис. 7.

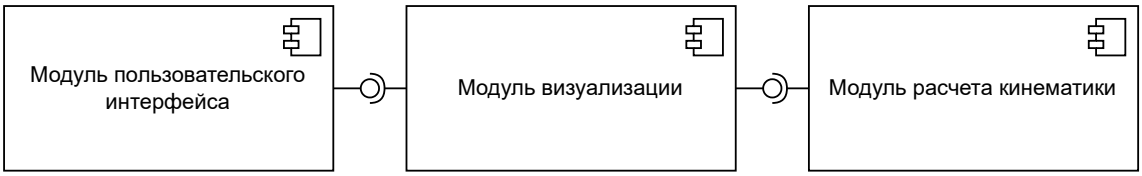

Рисунок 7 – Диаграмма компонентов

3.3 Реализация алгоритмов визуализации

Ключевым этапом визуализации является растеризация треугольников с использованием Z-буфера и интерполяцией нормалей для освещения Фонга.

В листинге 1 приведен метод растеризации, который принимает три вершины, прошедшие вершинные преобразования, и заполняет пиксели в буфере кадра.

```
1  void Renderer::rasterize(const RasterVertex& v0, const RasterVertex&
       v1, const RasterVertex& v2, const RenderSettings& settings) {
2      int minX = std::max(0, (int)std::min({v0.screenPos.x(),
           v1.screenPos.x(), ...}));
3      int maxX = std::min(width - 1, (int)std::max({v0.screenPos.x(),
           ...}) + 1);
4      int minY = std::max(0, ...}));
5      int maxY = std::min(height - 1, ...}) + 1);
6      float area = edgeFunction(v0, v1, v2);
7      float invArea = 1.0f / area;
8      for (int y = minY; y <= maxY; y++) {
9          uint32_t* scanline = (uint32_t*)frameBuffer.scanLine(y);
10         for (int x = minX; x <= maxX; x++) {
11             float w0 = edgeFunction(v1, v2, x, y) * invArea;
12             float w1 = edgeFunction(v2, v0, x, y) * invArea;
13             float w2 = 1.0f - w0 - w1;
14             if (w0 >= 0 && w1 >= 0 && w2 >= 0) {
15                 float z = w0 * v0.screenPos.z() + w1 *
                       v1.screenPos.z() + ...;
16                 if (zBufferTest(x, y, z)) {
17                     updateZBuffer(x, y, z);
18                     if (settings.isShadow) {
19                         scanline[x] = qRgb(30, 30, 30);
20                     } else {
21                         QVector3D normal = v0.normal * w0 + v1.normal
                               * w1 + ...;
22                         QVector3D worldPos = v0.worldPos * w0 +
                               v1.worldPos * w1 + ...;
23                         scanline[x] = calculatePhongPixel(worldPos,
                               normal, settings);
24                     }
25                 }
26             }
27         }
28     }
29 }
```

Листинг 1: Реализация алгоритма растеризации с Z-буфером

3.4 Реализация алгоритма кинематики

Для расчета вращения используется алгоритм обхода графа в ширину (BFS). Метод `updateKinematics` обновляет скорости и углы поворота всех шестеренок в цепи, исходя из параметров ведущей шестерни. Реализация представлена в листинге 2.

```
void Scene::updateKinematics() {
    for (auto& g : gears) {
        if (!g.isDriver) g.speed = 0;
        else g.speed = driverSpeed;
        g.isJam = false;
    }
    std::queue<int> q;
    q.push(driverIdx);
    visited[driverIdx] = true;

    while(!q.empty()) {
        int currIdx = q.front(); q.pop();
        Gear& curr = gears[currIdx];
        if (curr.isJam) continue;
        for (size_t i = 0; i < gears.size(); i++) {
            if (isConnected(curr, gears[i])) {
                float ratio = curr.radius / gears[i].radius;
                float newSpeed = -curr.speed * ratio;
                if (visited[i]) {
                    if (std::abs(gears[i].speed - newSpeed) > EPSILON)
                        {
                         curr.isJam = true;
                    }
                } else {
                    gears[i].speed = newSpeed;
                    visited[i] = true;
                    syncPhase(curr, gears[i]);
                    q.push((int)i);
                }
            }
        }
    }
}
```

Листинг 2: Реализация алгоритма расчета кинематики

3.5 Интерфейс пользователя

Графический интерфейс пользователя (GUI) разработан с использованием дизайнера форм Qt Designer и сохранен в файле `MainWindow.ui`. Интерфейс включает в себя:

1) Область рендеринга: занимает большую часть окна, отображает трехмерную сцену. Поддерживает управление камерой мышью (ЛКМ — вращение, колесо — масштабирование).

2) Панель управления: расположена слева и содержит:

   — выбор размера добавляемой шестерни;

   — режим удаления объектов;

   — переключатель режима предпросмотра;

   — управление ведущей шестерней: направление (по/против часовой) и слайдер скорости.

Обработка событий ввода реализована в классе `RenderWidget`. Метод `handleAction` использует класс `Camera` для построения луча из точки клика мыши и нахождения соответствующей ячейки гексагональной сетки.

Интерфейс главного окна изображен на рисунке 8.

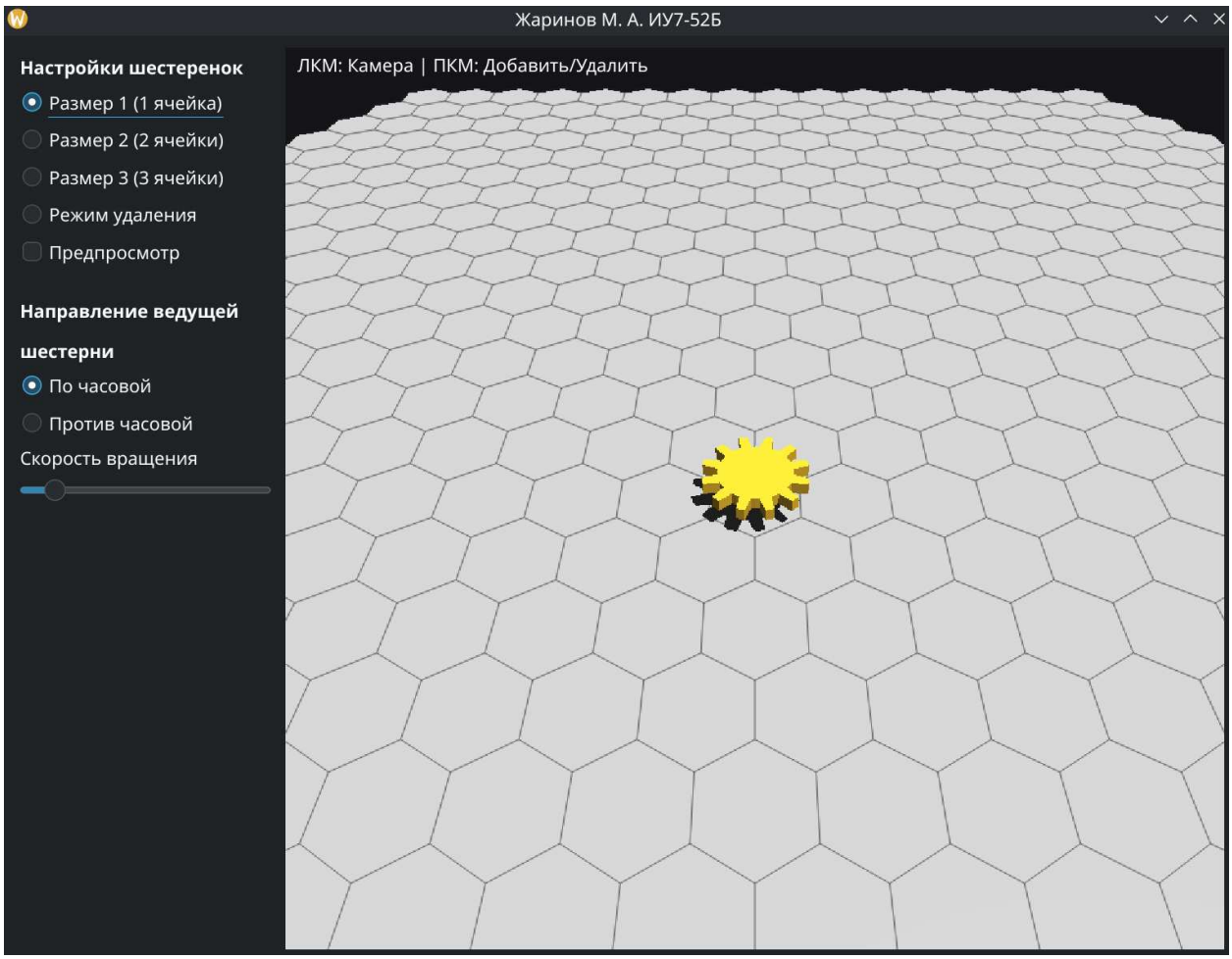

Рисунок 8 – Главное окно программы

Вывод

В данном разделе описаны средства реализации и архитектура программного комплекса. Приведены реализации основных алгоритмов: растеризации треугольника с учетом Z-буфера и освещения, а также распространения кинематических связей в системе шестеренок.

4 Исследовательская часть

4.1 Цель исследования

Целью исследования является оценка производительности разработанного алгоритма программной растеризации. Необходимо установить зависимость среднего времени отрисовки одного кадра от количества геометрических объектов (шестеренок) в сцене.

4.2 Технические характеристики

Исследование проводилось на ноутбуке со следующими характеристиками:

— операционная система — Debian Unstable GNU/Linux с ядром версии 6.17.12;

— процессор — AMD Ryzen 5 7535HS с 6 физическими ядрами и 12 логическими ядрами с максимальной тактовой частотой 4.60 ГГц [10];

— оперативная память — 16 Гбайт типа LPDDR5 в 4 каналах с тактовой частотой 6400 МГц.

4.3 Алгоритм проведения исследования

1) Приложение запускается в режиме тестирования производительности.

2) Камера фиксируется.

3) Производится замер времени выполнения функции отрисовки кадра для количества шестеренок на сцене от 2 до 100 с шагом 2.

4) Для каждого замера производится 100 повторений (отрисовок кадра), усредняется и сохраняется в файл.

4.4 Результаты исследования

В таблице 4 приведены полученные данные.

Таблица 4 – Зависимость времени рендеринга от количества объектов

N, шт.	T, мс	N, шт.	T, мс	N, шт.	T, мс
2	21.3832	36	32.0729	70	44.1520
4	22.2666	38	32.5122	72	44.5449
6	22.9500	40	33.4331	74	45.1821
8	23.7334	42	34.4030	76	45.7830
10	24.3168	44	35.3910	78	45.9031
12	25.4002	46	35.7086	80	46.9414
14	25.4836	48	36.0991	82	47.0862
16	26.3670	50	36.8994	84	47.7650
18	27.0504	52	37.7999	86	48.1849
20	27.5338	54	37.6583	88	48.7174
22	28.0172	56	38.1220	90	49.0622
24	29.1006	58	38.9347	92	50.0099
26	29.3729	60	39.4367	94	51.2052
28	30.0054	62	40.4037	96	52.6742
30	30.2218	64	41.5348	98	53.9166
32	30.6255	66	42.3975	100	54.8800
34	31.1867	68	43.3006		

На рисунке 9 представлен график зависимости времени кадра от количества шестеренок.

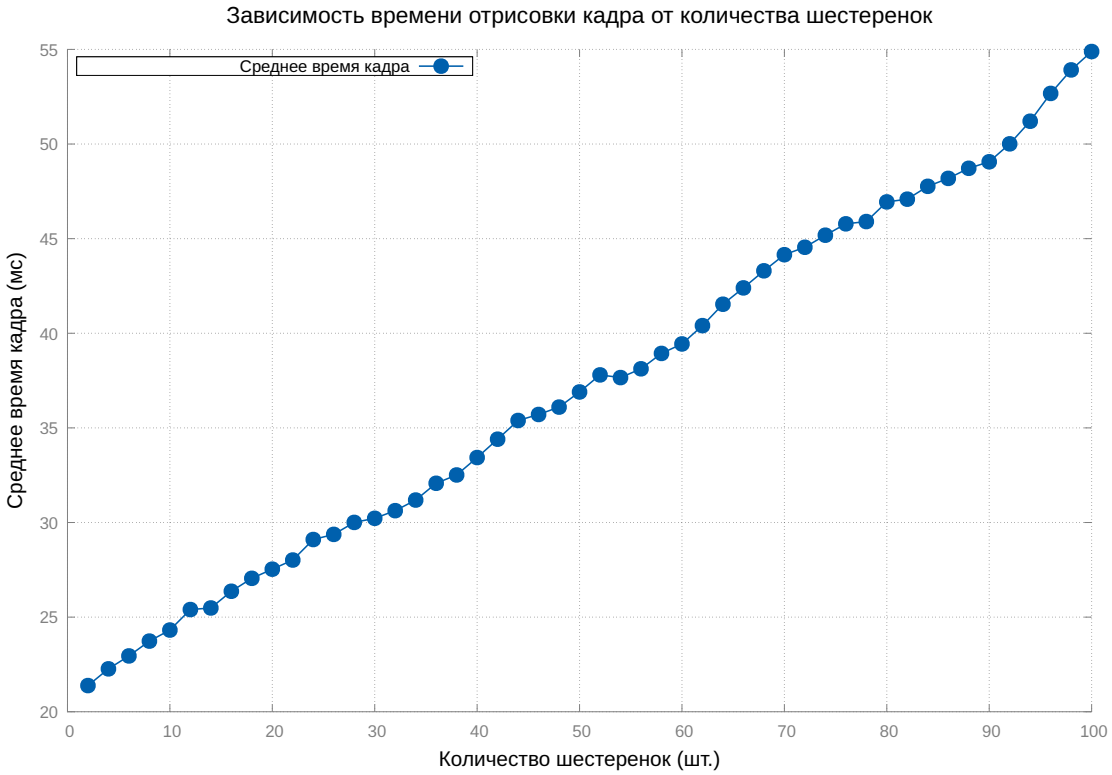

Рисунок 9 – График зависимости времени рендеринга от числа объектов

4.5 Анализ результатов

Зависимость времени рендеринга от количества объектов носит линейный характер. Это согласуется с теоретической оценкой сложности алгоритма растеризации и Z-буфера.

При увеличении количества объектов с 2 до 100 (рост в 50 раз), время кадра увеличилось с ≈ 21 мс до ≈ 55 мс (рост в 2.6 раза). Это свидетельствует о том, что значительную часть времени занимает отрисовка статического окружения и очистка буферов высокого разрешения, а добавление новой геометрии обрабатывается достаточно эффективно.

При 100 шестеренках время кадра составляет 54.8 мс, что соответствует частоте кадров ≈ 18 FPS, что обеспечивает достаточную интерактивность приложения.

Вывод

Проведенное исследование подтвердило линейную зависимость времени рендеринга от количества геометрических примитивов в сцене. Установлено, что разработанная программная реализация обеспечивает приемлемую производительность при нагрузке в 100 сложных объектов с учетом затрат на расчет теней и освещения на центральном процессоре. Полученные характеристики подтверждают эффективность выбранных алгоритмов и структур данных.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы была достигнута поставленная цель: разработано программное обеспечение для интерактивного конструирования и визуализации кинематики шестереночных механизмов в трехмерном пространстве.

В ходе работы были решены следующие задачи:

1) Выполнен анализ предметной области, в ходе которого были рассмотрены математические основы построения трехмерных изображений и методы моделирования механических связей.

2) Формализовано описание объектов сцены: определена геометрия зубчатых колес, осей и основания. Для позиционирования элементов механизма выбрана и математически описана осевая система координат гексагональной сетки, что позволило обеспечить корректное размещение сцепленных шестерен.

3) Обоснован выбор и осуществлена программная реализация базовых алгоритмов машинной графики. Для удаления невидимых поверхностей применен алгоритм Z-буфера, обеспечивающий корректное отображение пересекающейся геометрии. Реалистичность изображения достигнута за счет использования модели освещения Фонга и алгоритма планарных теней.

4) Разработан алгоритм расчета кинематики, основанный на представлении механизма в виде графа и его обходе в ширину. Алгоритм успешно вычисляет передаточные отношения, угловые скорости и направления вращения для всей цепи, а также детектирует ситуации механического заклинивания.

5) Спроектирована модульная структура программного обеспечения, выделяющая слои логики (сцена, кинематика) и представления (отрисовка, интерфейс). Приложение реализовано на языке C++ (стандарт C++20) с использованием библиотеки Qt.

6) Проведено исследование разработанного программного комплекса. Результаты показали, что время отрисовки кадра зависит линейно от количества объектов в сцене. Установлено, что разработанная программная реализация обеспечивает приемлемую производительность при нагрузке в 100 сложных объектов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Patel A. Hexagonal Grids [Электронный ресурс]. 2024. (Дата обращения: 20.11.2025). – URL: https://www.redblobgames.com/grids/hexagons/.

2. Артоболевский И. И. Теория механизмов и машин: Учеб. для втузов. 4-е изд., перераб. и доп. изд. М.: Наука, 1988. с. 640.

3. Foley J. D. Computer Graphics: Principles and Practice / J. D. Foley, A. van Dam, S. K. Feiner et al. 3rd Edition edition. Addison-Wesley Professional, 2013. p. 1264.

4. Роджерс Д. Алгоритмические основы машинной графики. М.: Мир, 1989. с. 512.

5. Шикин Е. В., Боресков А. В. Компьютерная графика. Полигональные модели. М.: ДИАЛОГ-МИФИ, 2005. с. 464.

6. Akenine-Möller T., Haines E., Hoffman N. Real-Time Rendering, Fourth Edition. Boca Raton: CRC Press, 2018. p. 1198.

7. Порев В. Н. Компьютерная графика. СПб.: БХВ-Петербург, 2002. с. 432.

8. C++ Reference [Электронный ресурс]. (Дата обращения: 20.11.2025). – URL: https://en.cppreference.com/w/.

9. Qt 6 Documentation [Электронный ресурс]. (Дата обращения: 20.11.2025). – URL: https://doc.qt.io/qt-6/index.html.

10. AMD. AMD Ryzen 5 7535HS. 2025. Электронный ресурс. – URL: https://www.amd.com/en/products/processors/laptop/ryzen/7000-series/amd-ryzen-5-7535hs.html.

ПРИЛОЖЕНИЕ А

К расчетно-пояснительной записке курсовой работы прилагаются следующие материалы:

1) презентация к курсовой работе, состоящая из 16 слайдов;

2) USB-флеш-накопитель с исходными кодами, электронными версиями РПЗ и презентации.